

Digital Pre-Registration App

Developer Guide for Proprietary OCR Engine Integration

1. Overview of the Application

The **Digital Pre-Registration App** is an offline-first client-side web application built for NSSF field staff. It enables fast, secure pre-registration of members by scanning Uganda National IDs or allowing manual data entry.

The application is built entirely using standard web technologies:

- **JavaScript (ES6+)**: Handles all the application logic, card image preprocessing (grayscale, contrast stretching, and canvas binarization filters), running the local WASM OCR engine, and regex parsing.
- **HTML5**: Defines the structure of the application, including the multi-tab layout, forms, camera inputs, and data tables.
- **CSS3 (Vanilla CSS)**: Manages all the styling, premium layouts, responsive design, dark-mode colors, and animations.

Currently, the application runs entirely in the browser using **Tesseract.js** (compiled in WebAssembly). To improve performance, reduce OCR errors (such as shadow or angle issues), and leverage corporate technology assets, this guide explains how a developer can merge a company-owned OCR tool into this codebase.

2. Image Processing & OCR Scan Workflow

Step 1 — Image Capture & Card Warping

When a photo of the ID card is loaded, the front-end uses HTML5 Canvas to crop and perspective-warp the card to a standard **856 × 540 pixel** container (10px/mm landscape standard).

Step 2 — Image Binarization (Filters)

The canvas is run through binarization, grayscale, and contrast-stretching filters locally to make the text stand out clearly from the background security watermarks.

Step 3 — Full-Page OCR Scan

The app executes the OCR engine on the entire warped card canvas. It extracts a single layout-independent text string of the front page.

Step 4 — Anchor-Based Semantic Parsing

The text block is passed to regex filters in `parser.js`. The app searches for anchor keywords (like "SURNAME" or "NIN") and extracts the text blocks positioned next to or immediately following those anchors.

3. Technical Codebase Structure

To implement the merge, your developer needs to know about three core files:

1. **index.html**: Contains the UI structure and includes the OCR libraries.
2. **js/app.js**: Houses the main application state and coordinates the scan trigger. The core function is `proceedWithWarpedImages(frontCanvas, backCanvas)` which triggers the OCR engine.
3. **js/parser.js**: Contains regex parsing functions like `parseFront(text)` and `parseBack(text)` which filter fields out of raw OCR blocks.

4. How to Merge a Proprietary OCR

Your developer can integrate the company's OCR engine in one of two ways, depending on how the tool is built:

Option A: Server-Side OCR API (Recommended)

If your company OCR runs on a central server, the developer can update `js/app.js` to send the captured canvas images via an API request. This removes the processing load from the local browser and guarantees high accuracy.

In `js/app.js`, replace the Tesseract scan call with a fetch request:

```
async function extractDataWithCompanyOCR(frontCanvas, backCanvas) { const formData = new
FormData(); // Convert canvas images to PNG Blobs const frontBlob = await new Promise(res
=> frontCanvas.toBlob(res, 'image/png')); const backBlob = await new Promise(res =>
backCanvas.toBlob(res, 'image/png')); formData.append('front_image', frontBlob,
'front.png'); formData.append('back_image', backBlob, 'back.png'); const response = await
fetch('https://your-company-ocr.com/api/extract', { method: 'POST', body: formData,
headers: { 'Authorization': 'Bearer YOUR_API_TOKEN' } }); const result = await
response.json(); // Directly populate the form fillForm({ surname: result.surname,
given_names: result.given_names, sex: result.sex, dob: result.dob, nin: result.nin,
nationality: result.nationality }); }
```

Option B: In-Browser JavaScript/WASM Library

If your company's OCR tool has a client-side JavaScript or WebAssembly wrapper, the developer can load the script in `index.html` and call it directly:

```
// In js/app.js, swap out Tesseract with your WASM engine: const text = await
CompanyOcrEngine.recognize(frontCanvas); const parsedData = parseFront(text);
```

```
fillForm(parsedData);
```

5. Expected Benefits

- **Increased Speed:** Server-based or optimized custom models can complete extraction in under 2 seconds (compared to 15+ seconds with general Tesseract.js).
- **Improved Accuracy:** A custom OCR model trained specifically on ID fonts and Uganda security backgrounds will drastically reduce typos and classification issues.